1. What is OOP (Object-Oriented Programming)?

Object-Oriented Programming is a way of structuring your code into **classes** and **objects**.

It focuses on:

- **Classes** → Blueprints or templates
- **Objects** → Real-world entities created from classes
- **Encapsulation** → Hiding data
- **Inheritance** → Reusing code
- **Polymorphism** → Using a single interface for different data types
- **Abstraction** → Hiding complex implementation details

🧱 **2. Class and Object**

A **class** defines the structure (attributes) and behavior (methods).
An **object** is an instance of a class.

Example:

```
# Example of Class and Object


class Car:
    # Constructor
    def __init__(self, brand, model):
        self.brand = brand   # attribute
        self.model = model   # attribute


    # Method
    def display_info(self):
        print(f"Car Brand: {self.brand}, Model: {self.model}")
```

```
# Create objects
car1 = Car("Toyota", "Corolla")
car2 = Car("Tesla", "Model S")

car1.display_info()  # Car Brand: Toyota, Model: Corolla
car2.display_info()  # Car Brand: Tesla, Model: Model S
```

📝 *__init__ is a special method (constructor) that runs automatically when you create an object.*

🪄 3. Encapsulation (Data Hiding)

It means restricting access to variables and methods from outside the class.

Example:

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance   # private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = Account("John", 5000)
acc.deposit(2000)
print(acc.get_balance())  # 7000
```

```
# print(acc.__balance)  # ❌ Error: cannot access private variable
```

📝 *Private attributes are written with double underscore __.*

🧬 4. Inheritance (Reusability)

A class can **inherit** properties and methods from another class.

Example:

```
class Animal:
    def speak(self):
        print("Animals make sounds")


class Dog(Animal):
    def speak(self):
        print("Dog barks")


class Cat(Animal):
    def speak(self):
        print("Cat meows")


dog = Dog()
cat = Cat()
dog.speak()  # Dog barks
cat.speak()  # Cat meows
```

📝 *Dog and Cat inherit from Animal but override the speak() method.*

🎭 5. Polymorphism

Means **many forms** — using the same function name but behaving differently for different classes.

Example:

```
class Bird:
    def fly(self):
        print("Most birds can fly")


class Penguin(Bird):
    def fly(self):
        print("Penguins cannot fly")


def flying_test(bird):
    bird.fly()


sparrow = Bird()
penguin = Penguin()


flying_test(sparrow)   # Most birds can fly
flying_test(penguin)   # Penguins cannot fly
```

📝 *Same function fly() behaves differently depending on the object type.*

🕊️ 6. Abstraction

Hiding internal details and showing only necessary parts.

Example using abc module:

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

rect = Rectangle(5, 10)
print(rect.area())  # 50
```

📝 *Abstract classes cannot be instantiated directly.*

🧠 7. Special Methods (a.k.a. Dunder Methods)

Python has built-in methods starting with double underscores __.

Example:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __str__(self):
```

```
        return f"{self.name} scored {self.marks}"


stu = Student("Alice", 95)
print(stu)  # Alice scored 95
```

📝 *__str__ defines how the object is represented when printed.*

✅ **Summary of OOP Concepts**

Concept	Description	Example
Class	Blueprint	class Car:
Object	Instance	car1 = Car()
Encapsulation	Hiding data	__balance
Inheritance	Code reuse	class Dog(Animal):
Polymorphism	Same name, different behavior	fly() in Bird and Penguin
Abstraction	Hiding details	abstractmethod